CYNEFORM GROUP

Backend Architecture

Gigs-hunting and BDP management system

Diagram set

NestJS, Drizzle, MariaDB: modules, layers, flows and security

Version 1.0 | November 2026

Status: design draft for review. No source code is produced in this document.

Contents of this part

1. Backend Architecture at a Glance

1.1 System context

2. Runtime and Deployment

3. Layers and Modules

4. Asynchronous Processing

5. Contract-First API

6. Key Backend Flows

7. Security Summary

8. MariaDB Design Notes

1. Backend Architecture at a Glance

The backend is a **modular monolith** built with NestJS and TypeScript, using Drizzle ORM on MariaDB. One codebase runs as two processes: an **API** that serves requests and a **worker** that runs background jobs. This document is the diagram set for that backend. It contains no code.

Table 1-1. Backend technology

Layer	Technology	Role
Language	TypeScript	One language across backend, frontend and shared contracts.
Framework	NestJS	Modules, dependency injection, guards, interceptors.
Validation	Zod (shared contracts package)	One definition of every request and response, used by both sides.
ORM	Drizzle ORM (MySQL dialect, mysql2 driver)	Typed schema and queries, explicit transactions, migrations.
Database	MariaDB 11.4 (Docker in development)	Relational data, row locking, JSON columns, UTC datetimes.
Queues and cache	Redis 7 with BullMQ	Jobs, schedules, sessions, rule cache, rate limits.
Files	S3-compatible storage (MinIO in development)	Evidence, training files, reports.
Security libraries	Helmet, throttler, cookie handling, password hashing	Headers, rate limiting, refresh cookie, credentials.
Delivery	Docker, Nginx, GitHub Actions, Lefthook	Containers, reverse proxy, CI, pre-push checks.
Repository	pnpm monorepo with shared catalog	Web app, API, worker and contracts share dependency versions.

1.1 System context

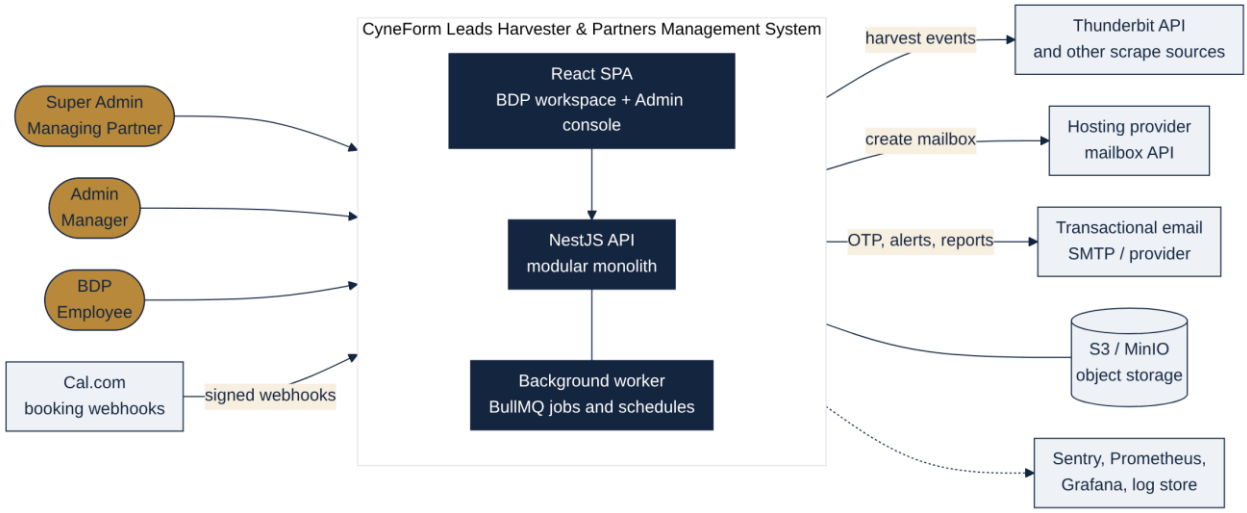


Figure 1-1. System context

2. Runtime and Deployment

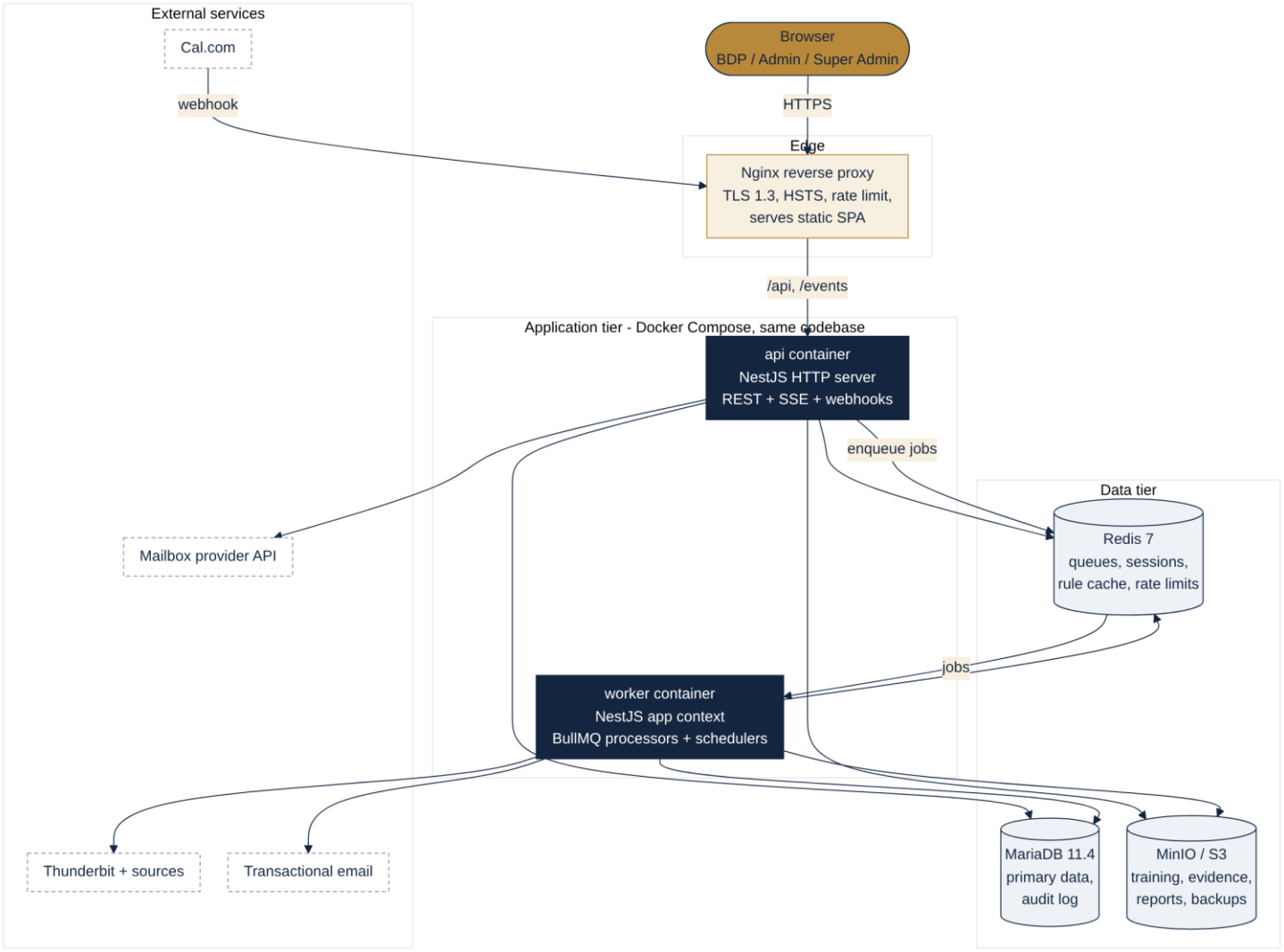


Figure 2-1. Runtime containers and connections

Table 2-1. Containers

Container	Responsibility
Nginx	TLS, static frontend, forwards /api to the API.
API	Handles requests, authentication, authorisation, business actions, webhooks, live event stream.
Worker	Runs queues and schedules: harvests, expiry, reminders, reports, mailbox requests, rule and bonus jobs.
MariaDB	System of record for all business data.
Redis	Queues, sessions, cached rules, rate limits.
Object storage	Files and generated documents.

3. Layers and Modules

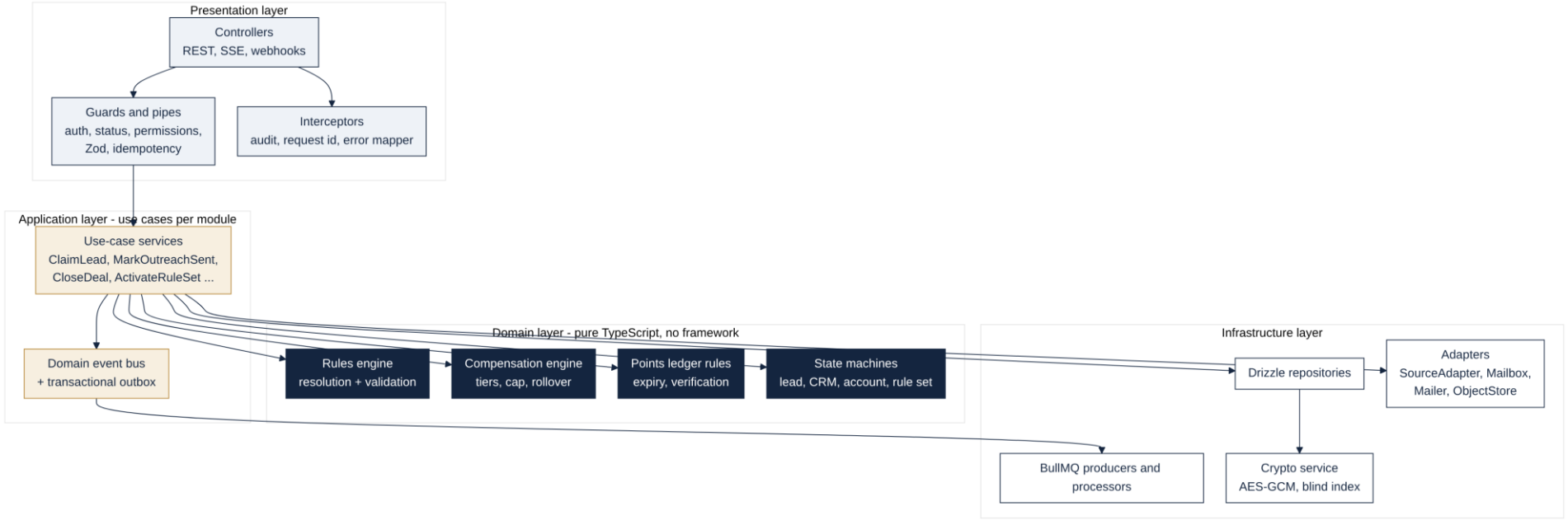


Figure 3-1. Layers and dependency direction

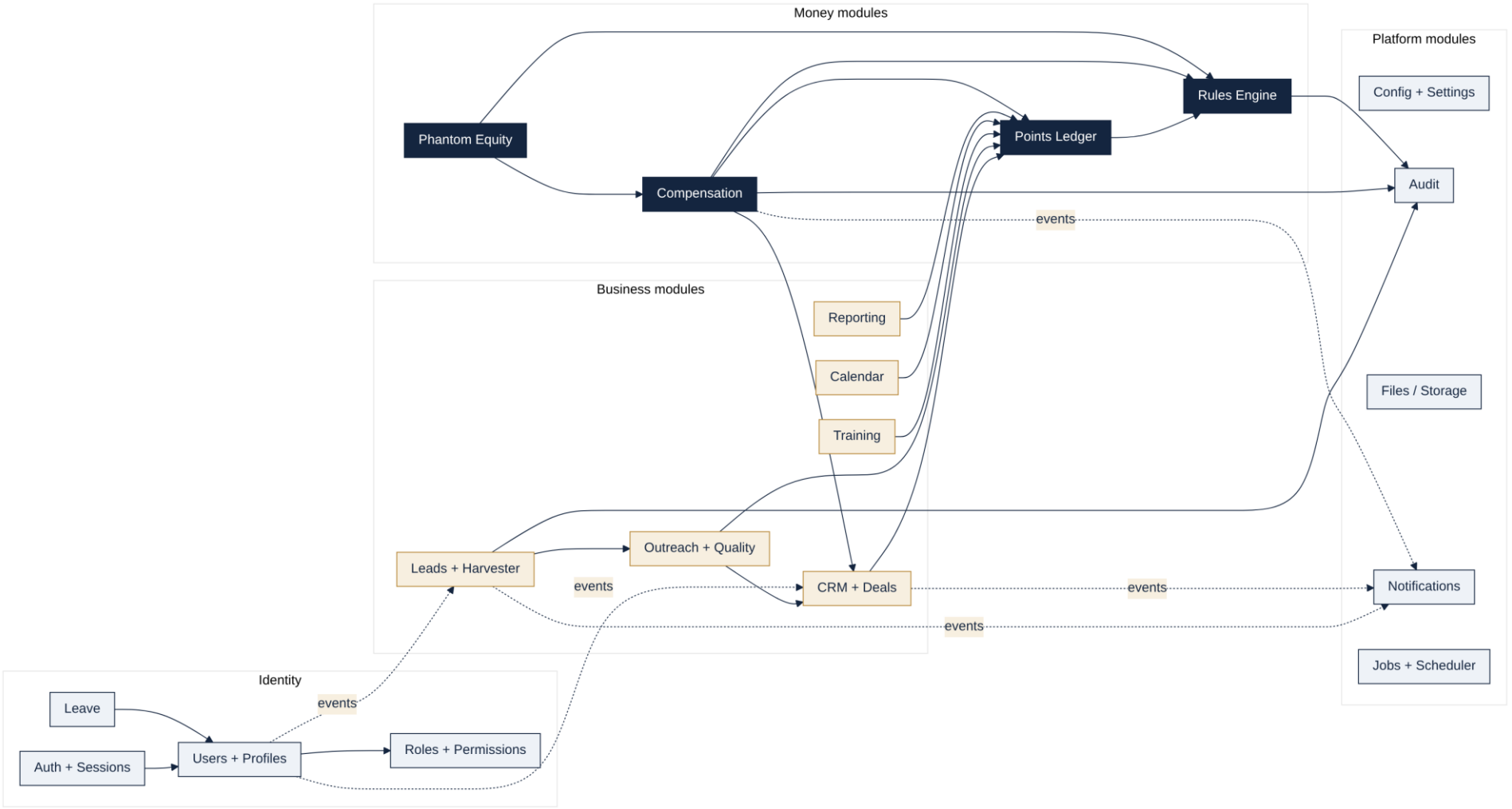


Figure 3-2. Backend module map

Table 3-1. Modules

Module	Responsibility
Identity and Access	Sign-in, sessions, roles, permissions, account lifecycle, leave.
Lead Harvester	Sources, harvest runs, lead pool, claims and timers.
Outreach and Quality	Outreach records, duplicate and generic detection, review queue.
CRM and Deals	Pipeline, follow-ups, evidence, bookings, deals, payments.
Points and Rules Engine	Versioned rules, simulation, append-only points ledger.
Compensation	Marginal commission, cap, rollover, performance bonus, metrics.
Phantom Equity	Grants, vesting, valuation marks, milestones, payouts.
Training and Calendar	Protected viewing, access rules, meetings, attendance.
Reporting and Notifications	Weekly and admin reports, notifications and delivery.
Platform	Files, audit log, settings, jobs, retention, data-subject requests.

4. Asynchronous Processing

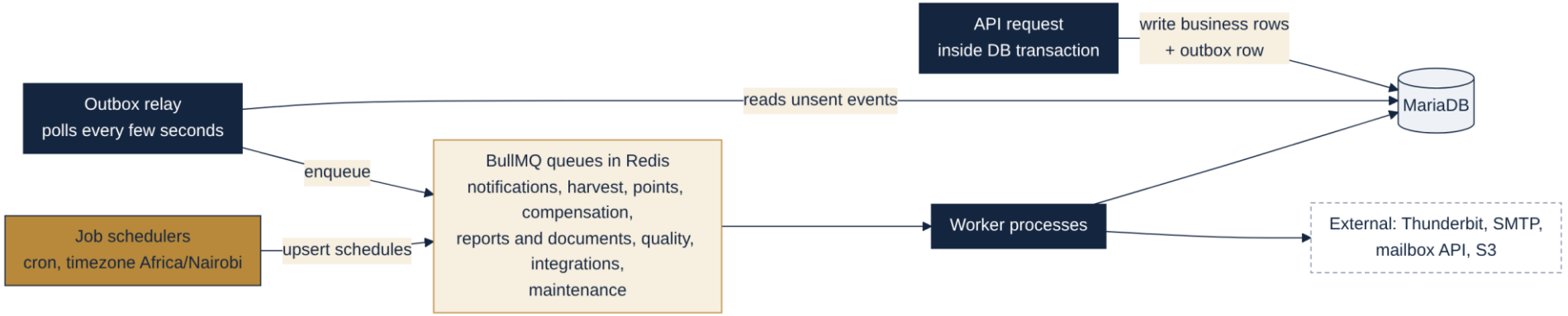


Figure 4-1. Queues, worker and scheduled jobs

Business writes and an outbox row are saved in one database transaction. A relay publishes outbox events to the queues, and the worker handles them with retries. This guarantees that an event is never lost even if the process stops between steps.

5. Contract-First API

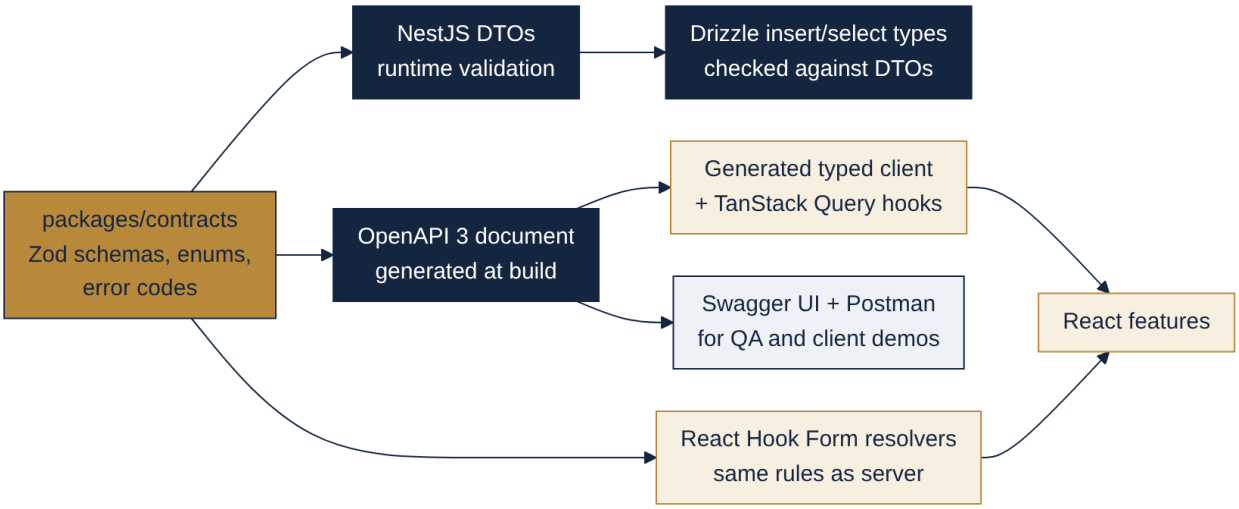


Figure 5-1. From shared contract to typed client

A change to a contract fails the build on whichever side is out of step, so the frontend and backend cannot drift apart.

6. Key Backend Flows

Each diagram shows the browser, API, cache, database and external services for one important flow.

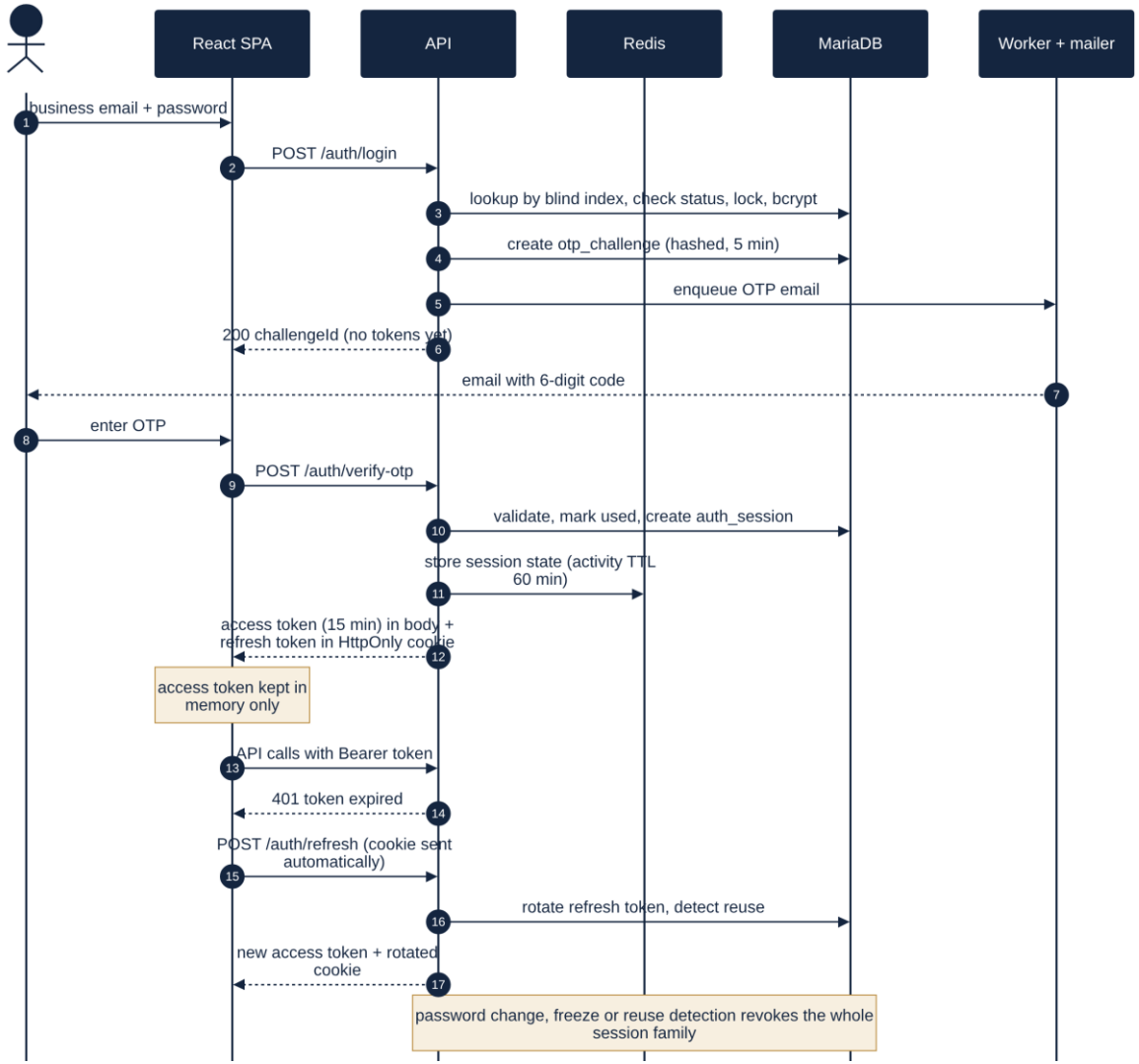


Figure 6-1. Sign-in with one-time code

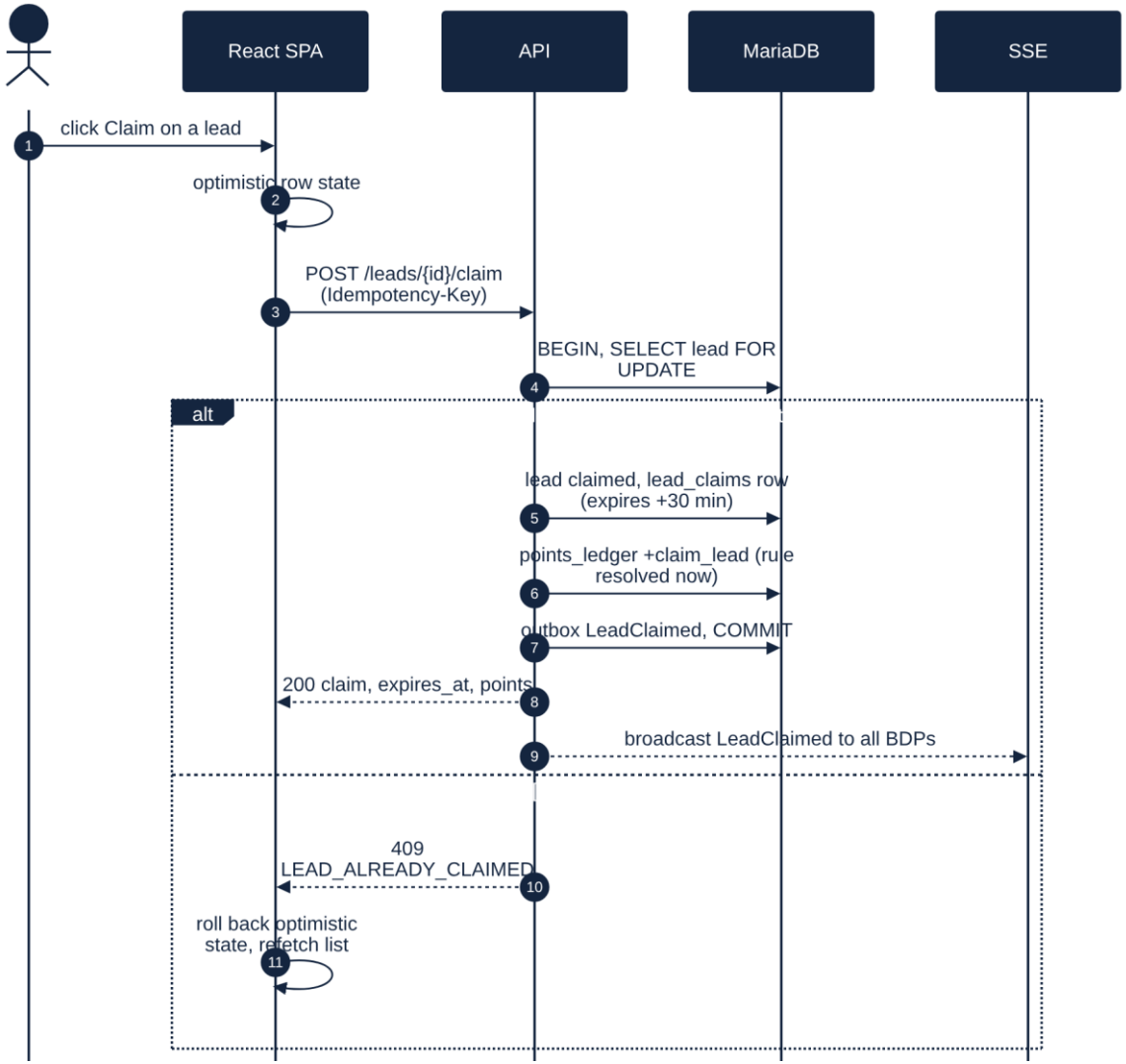


Figure 6-2. Claiming a lead (row lock and uniqueness)

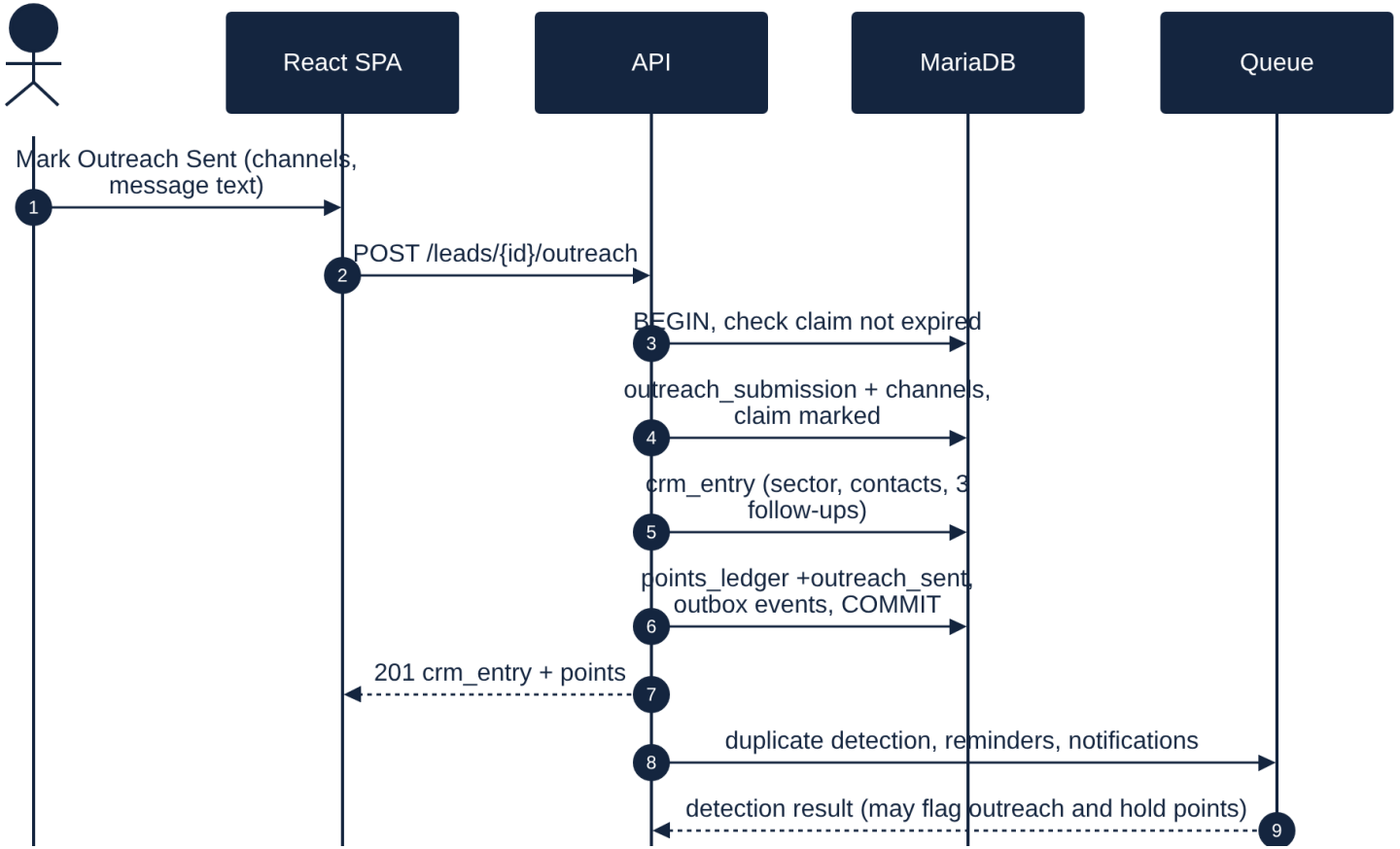


Figure 6-3. Recording outreach and quality check

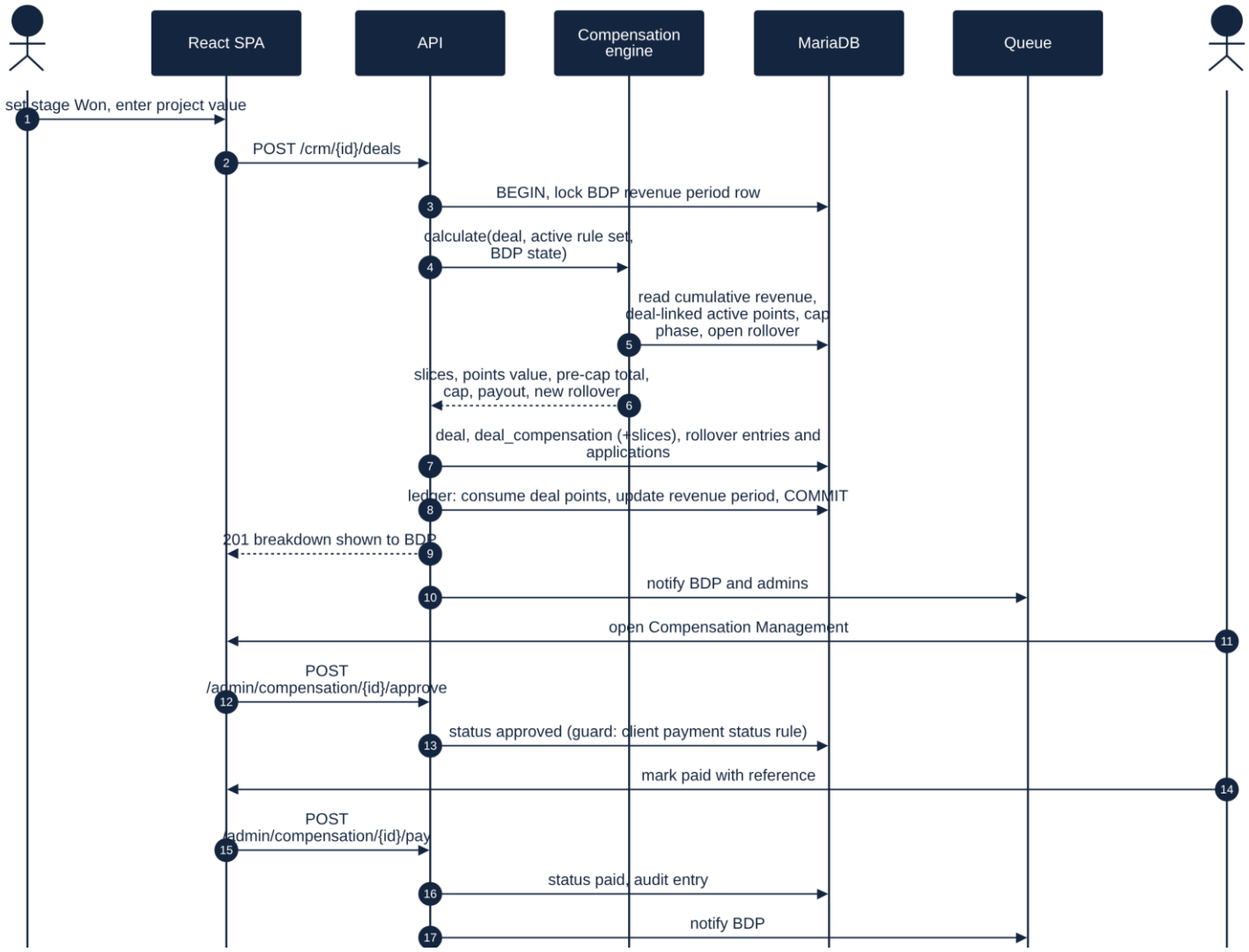


Figure 6-4. Closing a deal: commission, cap and rollover

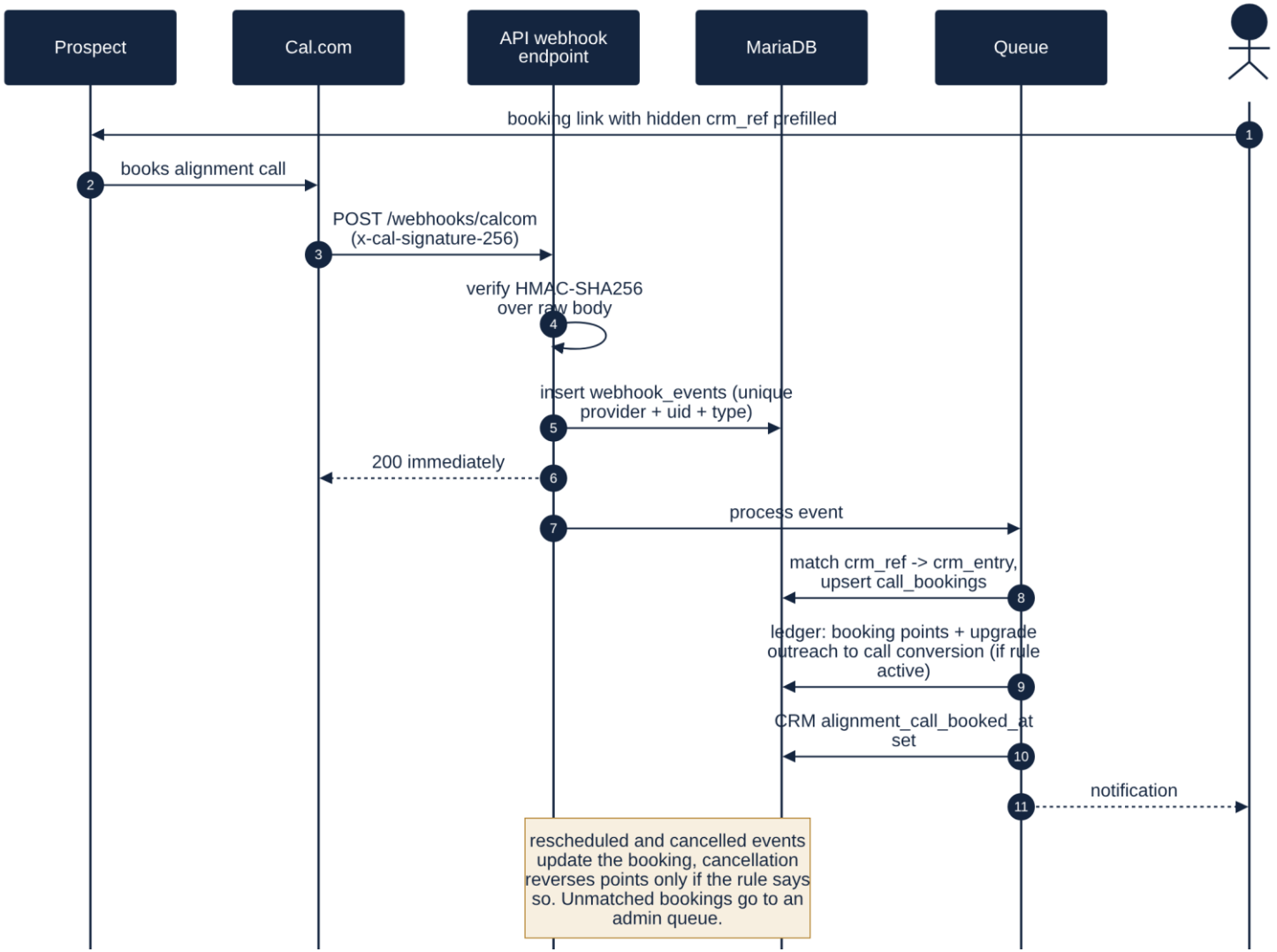


Figure 6-5. Cal.com booking webhook

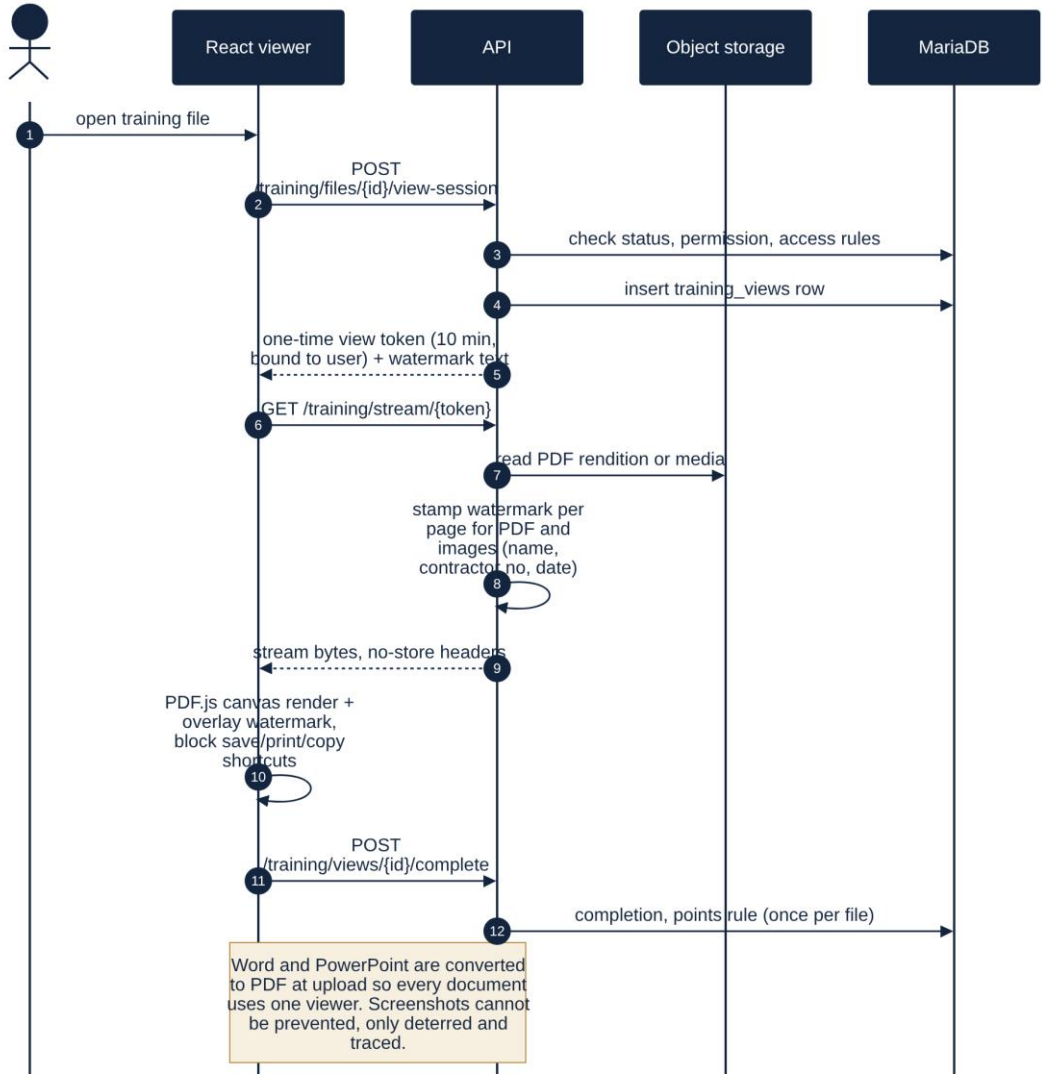


Figure 6-6. Protected training view

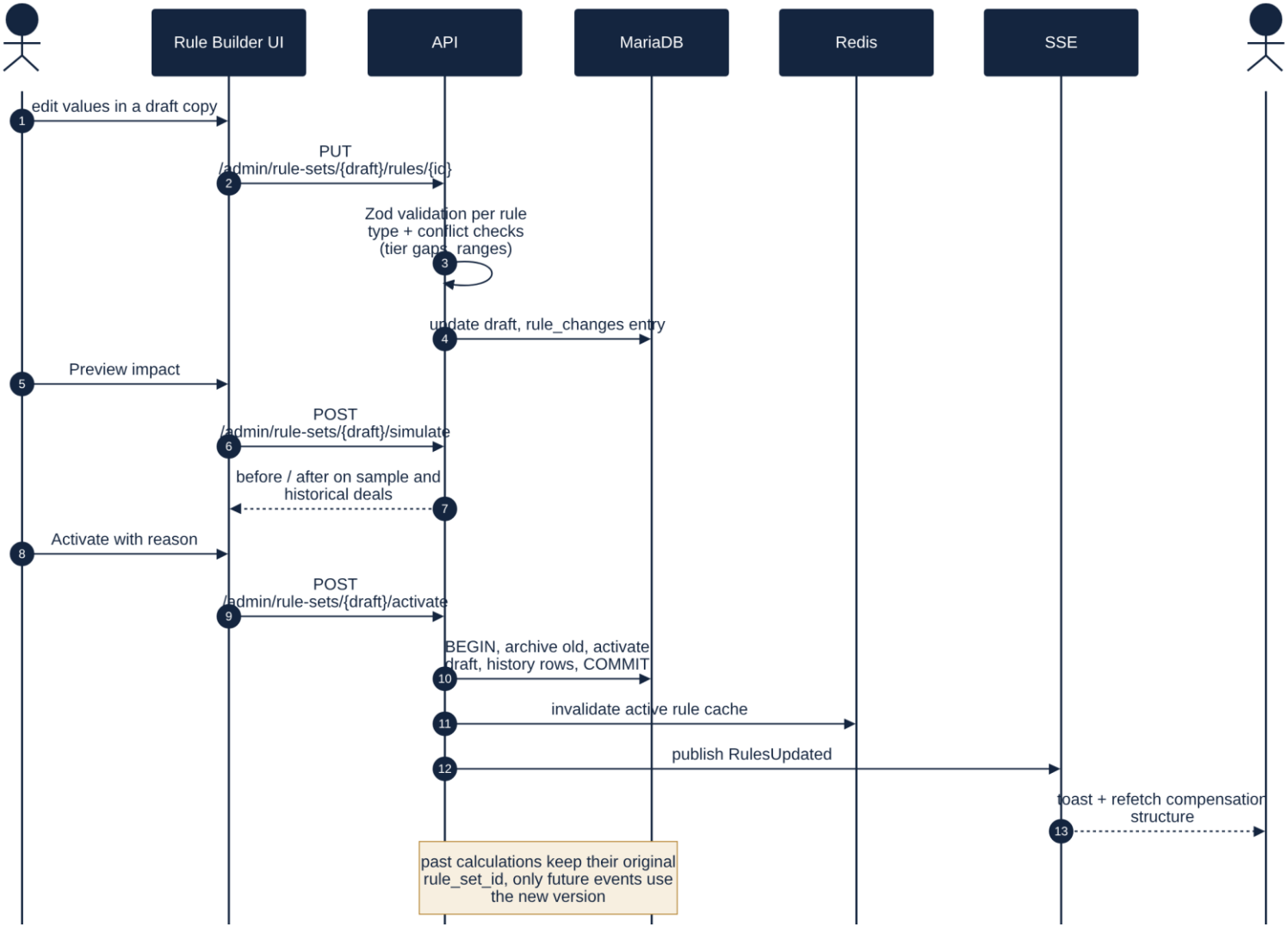


Figure 6-7. Rule change: draft, simulate, activate

7. Security Summary

Table 7-1. Security

Area	Design
Tokens	Access token 15 minutes, kept in memory. Refresh token 7 days in an HttpOnly cookie, rotated on every use with reuse detection.
Sessions	Maximum 3 per user, 60 minutes inactivity timeout.
Sign-in	Password plus 6-digit one-time code valid 5 minutes, hashed. Lockout after 5 failures in 15 minutes for 30 minutes.
Account setup	One-time set-password link. No temporary password.
Authorisation	Roles plus permissions with per-user overrides. Super-Admin-only actions cannot be delegated except where stated.
Data protection	Field encryption (AES-256-GCM) for sensitive fields, keyed hashes for searching, disk encryption at rest, TLS in transit.
Audit	Append-only audit log with hash chaining.
Webhooks	Signature checked on the raw body; every event stored first, then processed once.
Files	Training viewing through one-time tokens, server-side watermark, no download link.
Compliance	KDPA: 72-hour regulator notice, 48-hour processor notice, 30-day data-subject requests.

8. MariaDB Design Notes

These points adapt the earlier design to MariaDB.

Table 8-1. MariaDB notes

Topic	Approach in MariaDB
Identifiers	char(36) UUIDv7 generated in the application.
Times	datetime(3) in UTC; displayed in East Africa Time.
Money	decimal(15,2), returned as exact strings, never floating point.
Flexible data	JSON columns for rule values, raw payloads and breakdowns, validated by Zod before saving.
One active claim per lead	MariaDB has no partial unique index. A generated column holds the lead id while the claim is active and NULL otherwise, with a unique index on it. Row locking on the lead guards the decision.
Inserted row read-back	The ORM does not return inserted rows for this dialect, so the application knows the id up front and reads back by id.
Append-only tables	Points ledger, audit log and commission records are never updated or deleted by the application; database permissions enforce it.
Migrations	Generated by the ORM tooling, reviewed, and applied before each release.